

abc - CNN Supplement

Separate from the six-model group comparison

Design and complete pipeline

Raw audited RGB images -> common grouped holdout -> development train/validation split -> training-only horizontal/vertical flips -> normalization to $[-1, 1]$ -> two Adam configurations -> validation checkpoint selection and early stopping -> holdout evaluation -> saved model -> single-image prediction.

The network has four convolution blocks (32, 64, 128, 256 channels), each with BatchNorm, ReLU and max pooling. A 16,384-to-256 dense layer and two output logits follow dropout 0.4 and 0.3. Images are resized to 128 x 128. Class-weighted cross entropy addresses class frequency differences.

Training / validation / holdout images: 599 / 300 / 225. Both classical and CNN holdouts were checked by image path, label and file hash. CNN uses fewer training images, a single validation split and a different representation, so scores are supplementary rather than a controlled architecture-only comparison.

Tuning evidence

Trial	Learning rate	Weight decay	Epochs run	Best val F1
1	0.001	0.0001	16	0.9658
2	0.0003	0.001	16	0.9763

At most 16 epochs per trial; patience 4 without validation F1 improvement. ReduceLROnPlateau halves learning rate after two stagnant epochs. Best validation checkpoint is restored before test evaluation. Selected initial learning rate 0.0003, weight decay 0.001. Total tuning time 1192.5 seconds.

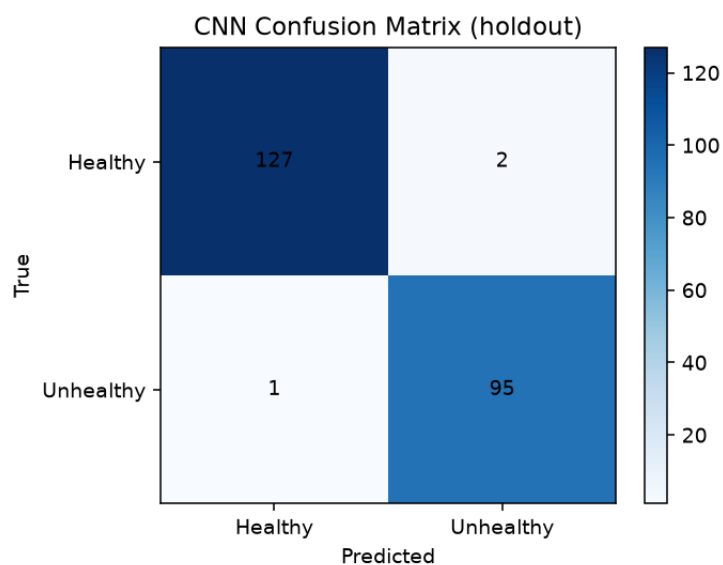
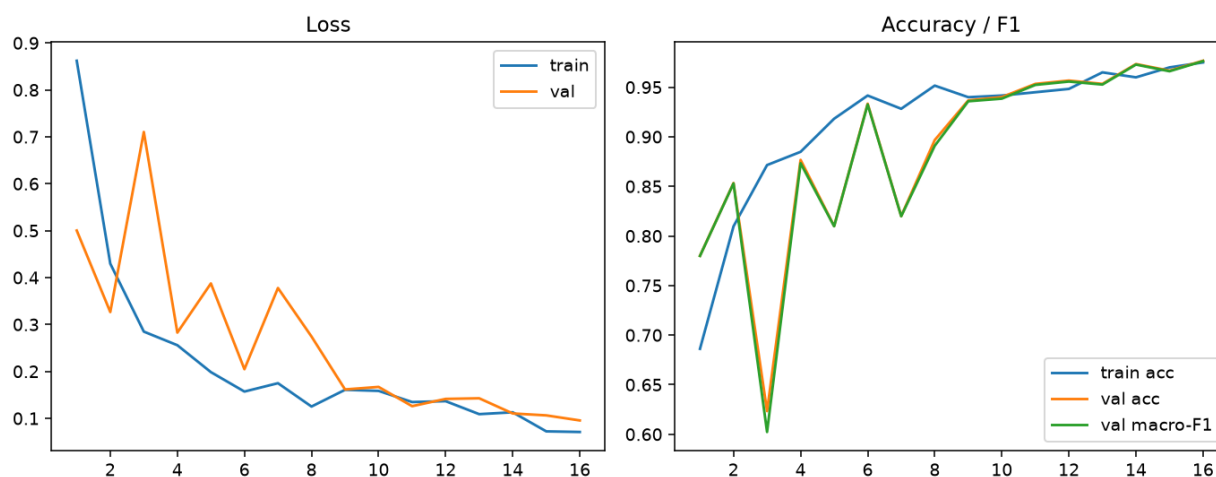
Reproduce: `python CNN/train_cnn.py`

Predict: `python predict.py IMAGE --model cnn`

Evidence: `CNN/results/tuning_results.csv` and `CNN/analytics/all_trials_history.csv`.

CNN evaluation and visualizations

Metric	Measured value
Holdout accuracy	98.67%
Macro-F1	0.9864
Macro precision / recall	0.9858 / 0.9870
95% group-bootstrap F1	0.9693 - 1.0000



Plots show the winning trial history and the restored checkpoint's holdout confusion matrix. Rows are true labels; columns are predictions. Bootstrap resampling uses perceptual groups (1,000 resamples).

CNN interpretation, limitations and transparency

The selected CNN achieves 98.67% holdout accuracy and 0.9864 macro-F1. It makes 1 false-healthy predictions and 2 false-unhealthy predictions.

Comparison context

The classical model selected by grouped CV is Gradient Boosting, with holdout macro-F1 0.9864. This contextual result is kept separate from the six-model ranking. Differences may reflect representation, training-set size, optimization, regularization and split variability; no significance claim is made.

Limitations and improvement proposals

Two settings are a small search, and early stopping may miss later improvements. Training uses a single seed and validation partition. The same previously evaluated holdout is reused. Perceptual groups are not verified plant IDs. A CNN can memorize backgrounds or capture conditions. Flips do not reproduce all field variation, and confidence is not calibrated.

Collect external, plant-disjoint images before claiming deployment readiness. Additional seeds, wider development-only searches, segmentation and transfer learning are possible future experiments, not completed work.

AI transparency and references

Codex assisted implementation, tuning orchestration, artifact checks and report drafting. Values are computed from saved predictions; group members must review and understand the work. Dataset attribution and course briefs are listed in the main group report. Implementation uses the installed PyTorch version 2.14.0+cpu. Architecture and optimization settings are recorded in CNN/train_cnn.py and CNN/results/cnn_metrics.json.

Saved output

CNN/outputs/cnn_model.pt stores the state dictionary, class order, image size and architecture identifier. predict.py recreates the same preprocessing and network before inference. Uploaded image content is not a model artifact and is never deserialized as weights.